Production App Reverse-Engineering Playbook

> *A repeatable workflow for rebuilding a production-grade application from a real production app as reference.*

Core Principle

Do not rebuild screens. Rebuild the system that makes those screens possible.

The overall journey:

```
REFERENCE APP
     ↓
1. OBSERVE
     ↓
2. MAP
     ↓
3. TRACE USER JOURNEYS
     ↓
4. DISCOVER DOMAIN
     ↓
5. DISCOVER STATE & RULES
     ↓
6. DESIGN SYSTEM
     ↓
7. BUILD FOUNDATION
     ↓
8. BUILD FIRST VERTICAL SLICE
     ↓
9. VALIDATE ARCHITECTURE
     ↓
10. EXPAND
     ↓
11. HARDEN
     ↓
12. FINISH
```

PHASE 0 — DEFINE THE CHALLENGE

Before touching the codebase, define what you're trying to reproduce.

Reference

```
App:
Platform:
Purpose:
Why I chose it:
```

Scope

```
I WILL reproduce:
-

I WILL NOT reproduce:
-

Target quality:
-

Definition of done:
-
```

Important

The goal is **not**:

```
Recreate the original source code.
```

The goal is:

```
Recreate the important behavior,
architecture properties,
reliability,
state management,
and production qualities.
```

PHASE 1 — OBSERVE

Goal

Understand what the reference application actually does.

Do not design your architecture yet.

You are a detective.

Explore the application systematically.

Record

Screens / destinations

```
-
-
-
```

User actions

```
-
-
-
```

Navigation

```
A → B
B → C
C → D
```

Loading states

```
-
-
```

Empty states

```
-
-
```

Error states

```
-
-
```

Persistence behavior

```
-
-
```

Network behavior

```
-
-
```

Permissions

```
-
-
```

Notifications / background behavior

```
-
-
```

For every important behavior

Record:

```
Trigger:
User action:
Visible result:
Data affected:
Next state:
Side effects:
Failure behavior:
Unknowns:
```

Separate knowledge into 3 categories

OBSERVED

Something you actually verified.

```
Example:

Observed:
After adding a meal, today's nutrition total changes.
```

HYPOTHESIS

Something you believe is happening internally but have not verified.

```
Hypothesis:
The meal is persisted locally before the server request finishes.
```

UNKNOWN

Something you don’t know yet.

```
Unknown:
Does the app queue the change when offline?
```

Rule

Never turn an **unknown** into an architectural assumption without investigating it.

Phase 1 is complete when

```
[ ] Major screens explored
[ ] Major user actions explored
[ ] Major workflows discovered
[ ] Loading states recorded
[ ] Empty states recorded
[ ] Error states recorded
[ ] Persistence behavior investigated
[ ] Network behavior investigated
[ ] Important unknowns documented
```

PHASE 2 — MAP THE SYSTEM

Goal

Turn observations into a map of **capabilities**.

Do NOT create code modules yet.

Do NOT automatically make every screen a module.

Example

Instead of:

```
HomeScreen
FoodScreen
AnalysisScreen
SocialScreen
ProfileScreen
```

think:

```
ACCOUNT
 ├── Authentication
 ├── Profile
 └── Settings

NUTRITION
 ├── Food
 ├── Food Search
 ├── Meal Logging
 ├── Daily Nutrition
 └── Nutrition History

ANALYSIS
 ├── Daily Analysis
 ├── Trends
 └── Recommendations

SOCIAL
 ├── Feed
 ├── Posts
 ├── Comments
 ├── Likes
 └── Following
```

These are **capabilities**, not necessarily final modules.

Find relationships

Ask:

> *What depends on what?*

Example:

```
Food Logging
      ↓
Meal
      ↓
Daily Nutrition
      ↓
Analysis
      ↓
Recommendations
```

And:

```
User
 ├── Meals
 ├── Nutrition records
 ├── Posts
 └── Social relationships
```

Phase 2 output

Create:

```
system-map.md
```

Containing:

```
Capabilities
Relationships
Major dependencies
Potential domains
Unknown dependencies
```

PHASE 3 — TRACE USER JOURNEYS

Goal

Understand complete workflows instead of isolated screens.

A production system is usually a collection of workflows.

Example

Instead of:

```
Food screen
```

trace:

```
Open app
 ↓
Login
 ↓
Home
 ↓
Search food
 ↓
Select food
 ↓
Enter amount
 ↓
Add meal
 ↓
Nutrition changes
 ↓
Analysis changes
 ↓
Recommendation may change
```

Journey template

```
# Journey: ___________________

Goal:

Starting state:

1.
2.
3.
4.
5.

Data created:

Data modified:

Other features affected:

Side effects:

Success state:

Failure states:

Unknowns:
```

Important question

For every action ask:

> ***“What else in the application should change because of this?”***

This reveals hidden dependencies.

PHASE 4 — DISCOVER THE DOMAIN

Goal

Find the important **business concepts** behind the UI.

Do not start with screens.

Start with nouns and concepts.

Example

For a diet application you might discover:

```
User
Food
Meal
FoodEntry
Nutrition
DailyRecord
Goal
Recommendation
Post
Comment
Like
Follow
```

This is only an example.

Derive your actual model from observation.

Entity template

For each important entity:

```
Entity:

Purpose:

Important fields:

Owned by:

Created by:

Modified by:

Read by:

Related entities:

Lifecycle:

Important rules:
```

Example

```
Entity: FoodEntry

Purpose:
Represents a food consumed by a user.

Related to:
User
Meal
Food

May affect:
Meal totals
Daily nutrition
Analysis
Recommendations
```

PHASE 5 — DISCOVER STATE & RULES

Goal

Understand how important things change over time.

This is where you begin discovering the real system.

For every important entity/workflow ask:

```
What states can it have?

What causes state changes?

Who can change it?

What rules must always remain true?

What other data changes because of it?

What happens if the operation fails?
```

Example

```
FoodEntry

Not Logged
    ↓
Logged
    ↓
Edited
    ↓
Deleted
```

And:

```
FoodEntry added
      ↓
Meal updated
      ↓
Daily nutrition updated
      ↓
Analysis updated
      ↓
Recommendation may change
```

Find invariants

An invariant is something that should always remain true.

Examples:

```
A deleted meal should not contribute to nutrition totals.

A user's meal must belong to that user.

A nutrition total must equal the relevant entries.

A logged-out user must not access private data.
```

Write them down.

PHASE 6 — DESIGN THE SYSTEM

IMPORTANT

Only now start making serious architecture decisions.

You have:

```
Observed behavior
+
Capabilities
+
User journeys
+
Domain concepts
+
State
+
Rules
```

Now you can design your implementation.

6.1 Define domains

Example:

```
Account
Nutrition
Analysis
Recommendations
Social
```

But don’t force these names.

Use boundaries that make sense for the actual system.

6.2 Define ownership

For every important piece of state:

```
Who owns it?

Who can modify it?

Who can read it?

What is the source of truth?

What happens when it becomes stale?
```

6.3 Define data flow

For example:

```
UI
 ↓
Presentation
 ↓
Domain
 ↓
Repository / Data Boundary
 ↓
Local DB / Network
```

The exact architecture can differ.

The important thing is that the dependencies are intentional.

6.4 Decide persistence

For each important data type:

```
Memory only?
Local database?
Remote server?
Cache?
Both local + remote?
```

6.5 Decide synchronization

Ask:

```
What happens offline?

What happens when local data differs from server?

Can changes be queued?

Can operations be retried?

Can operations happen twice?

Which side is authoritative?
```

6.6 Decide error handling

Identify:

```
Network unavailable
Timeout
Unauthorized
Forbidden
Not found
Validation error
Conflict
Server error
Malformed data
Local database failure
Unknown failure
```

Don’t make every error behave identically.

6.7 Decide lifecycle behavior

Ask:

```
What happens if the screen disappears?

What happens if the app goes background?

What happens if Android kills the process?

What survives restart?
```

PHASE 7 — BUILD THE FOUNDATION

Goal

Build only the infrastructure required by your architecture.

Potential foundation:

```
Project structure
Dependency injection
Networking
Persistence
Authentication/session
Configuration
Error model
Logging
Testing infrastructure
```

DO NOT

Spend weeks building:

```
A giant framework
A huge Common module
Every possible abstraction
Infrastructure for features you haven't understood
```

Build the minimum solid foundation required for your first real workflow.

PHASE 8 — BUILD ONE VERTICAL SLICE

This is the most important implementation rule.

Do NOT build:

```
All screens
 ↓
All ViewModels
 ↓
All repositories
 ↓
All APIs
```

Instead build:

```
ONE COMPLETE USER JOURNEY
```

Example

```
Login
 ↓
Home
 ↓
Search food
 ↓
Select food
 ↓
Record meal
 ↓
Daily nutrition updates
```

Build that complete path:

```
UI
 ↓
Presentation
 ↓
Domain
 ↓
Data
 ↓
Network / Database
 ↓
Persistence
 ↓
Error handling
 ↓
Tests
```

That is a **vertical slice**.

PHASE 9 — STOP AND VALIDATE

This is where your old workflow failed.

After the first meaningful slice:

STOP.

Do not immediately implement 30 more features.

Ask:

```
Where does this data belong?

Who owns it?

What is the source of truth?

Can multiple screens consume it?

Can business rules change without rewriting UI?

Can I add another item without hacks?

Can I change the calculation without modifying unrelated features?

What happens if network fails?

What happens if the app is killed?

What happens if the same action occurs twice?
```

Architecture red flags

If adding a small feature requires:

```
Changing many unrelated screens
Duplicating state
Adding global variables
Adding special flags
Creating circular dependencies
Copying business logic
Adding "temporary" hacks
```

STOP.

Fix the architecture now.

PHASE 10 — EXPAND

Once your first slices prove that the architecture works, continue.

Example:

```
Slice 1
Authentication → Home → Food Logging

Slice 2
Daily Nutrition → History

Slice 3
Analysis → Trends

Slice 4
Recommendations

Slice 5
Social Feed → Posts → Comments

Slice 6
Profile → Settings
```

Each slice goes through:

```
Understand
 ↓
Design
 ↓
Implement
 ↓
Test
 ↓
Validate architecture
 ↓
Refactor if needed
 ↓
Next slice
```

PHASE 11 — HARDEN

Now deliberately try to break your application.

Network

```
[ ] No network
[ ] Slow network
[ ] Timeout
[ ] Server error
[ ] Retry
[ ] Duplicate request
```

Lifecycle

```
[ ] Background app
[ ] Return to app
[ ] Process death
[ ] App restart
[ ] Configuration change
[ ] Navigation away and back
```

Data

```
[ ] Empty data
[ ] Missing data
[ ] Invalid data
[ ] Large data
[ ] Stale data
[ ] Database failure
[ ] Migration
```

Concurrency

```
[ ] Repeated taps
[ ] Multiple requests
[ ] Out-of-order responses
[ ] Conflicting updates
[ ] Stale state
```

Authentication

```
[ ] Expired session
[ ] Logout
[ ] Re-login
[ ] Unauthorized request
```

UI

```
[ ] Loading
[ ] Empty
[ ] Error
[ ] Retry
[ ] Offline
[ ] Accessibility
```

PHASE 12 — FINISH

Final pass:

```
[ ] Performance
[ ] Security
[ ] Accessibility
[ ] Crash handling
[ ] Logging
[ ] Analytics
[ ] Database migrations
[ ] Network reliability
[ ] Testing
[ ] Release configuration
[ ] Secrets/configuration
[ ] UI polish
```

Then ask:

> ***"If I had to add a significant feature tomorrow, would I know exactly where it belongs?"***

If not, the project isn't finished.

THE CORE DEVELOPMENT LOOP

After the initial discovery:

```
UNDERSTAND
    ↓
DESIGN
    ↓
IMPLEMENT ONE VERTICAL SLICE
    ↓
TEST
    ↓
OBSERVE WHAT YOU LEARNED
    ↓
REFINE ARCHITECTURE
    ↓
NEXT SLICE
```

Never:

```
SCREEN
 ↓
CODE
 ↓
NEXT SCREEN
 ↓
PATCH
 ↓
NEXT SCREEN
 ↓
JUGAAD
 ↓
REWRITE
```

IMPORTANT RULES

Rule 1 — Screens are evidence, not architecture

A screen tells you what the user sees.

It does not automatically tell you:

```
Where data belongs
Who owns state
What the domain is
How persistence works
What dependencies should exist
```

Rule 2 — Understand before abstracting

Don’t create abstractions because:

> *“We might need this later.”*

Create them because you understand the actual problem they solve.

Rule 3 — One source of truth

Ask:

> *“What is the authoritative state?”*

Avoid having the same business state independently maintained by:

```
Screen A
Screen B
ViewModel
Repository
Singleton
```

Rule 4 — Business rules should have an owner

Do not duplicate the same business rule across multiple screens.

Rule 5 — Build vertically

Prefer:

```
One complete workflow
```

over:

```
All UI first
All data later
All backend later
```

Rule 6 — Validate architecture early

A bad architectural decision discovered after 5 features is cheap.

The same decision discovered after 50 features is expensive.

Rule 7 — Complexity is not the goal

Do not reproduce complexity merely because the reference app has complexity.

Ask:

```
Is this complexity required by behavior?

Is it required for reliability?

Is it required for scale?

Is it required by the domain?

Or is it merely accidental implementation complexity?
```

Reproduce **necessary complexity**, not unnecessary complexity.

PROJECT FILE STRUCTURE

For each reference application, maintain:

```
reference-project/
│
├── README.md
│
├── 01-observation.md
├── 02-system-map.md
├── 03-user-journeys.md
├── 04-domain-model.md
├── 05-state-and-rules.md
├── 06-architecture.md
│
├── 07-adrs/
│   ├── 001-data-ownership.md
│   ├── 002-sync-strategy.md
│   └── ...
│
├── 08-vertical-slices.md
├── 09-validation.md
├── 10-hardening.md
│
└── unknowns.md
```

UNKNOWNS FILE

Maintain this throughout the entire project.

Example:

```
# Unknowns

## Question

What happens when the user edits a meal while offline?

## Evidence

Not yet tested.

## Hypothesis

The change may be queued locally.

## Confidence

Low.

## How to verify

Disable network → edit meal → kill app → reopen → restore network.
```

Never silently turn guesses into architecture.

ARCHITECTURE DECISION RECORD

For important decisions:

```
# ADR: Daily Nutrition Ownership

## Context

Multiple screens display daily nutrition.

## Decision

The Nutrition domain owns the calculation and state.

## Why

Prevents duplicated calculations and inconsistent values.

## Alternatives

1. Each screen calculates totals.
2. Shared UI state.
3. Nutrition domain owns calculation.

## Consequences

Positive:
- One source of truth.
- Easier testing.
- Easier future changes.

Negative:
- Requires explicit domain boundary.
```

Use ADRs only for meaningful decisions.

DEFINITION OF DONE

The application is NOT done merely because:

```
All screens exist.
```

It is done when:

```
[ ] Major behavior works
[ ] Domain model is coherent
[ ] Data ownership is clear
[ ] State transitions are understood
[ ] Failure behavior is handled
[ ] Lifecycle is handled
[ ] Concurrency is considered
[ ] Important behavior is tested
[ ] Architecture is maintainable
[ ] New features can be added without hacks
[ ] App is performant enough
[ ] App is secure enough
[ ] App is releasable
```

THE QUESTION TO ASK YOURSELF

Whenever you feel:

> *“I know this screen. Let me build it.”*

STOP.

Ask:

```
What user journey is this part of?

What business concept does it represent?

Who owns the data?

What is the source of truth?

What else changes when this happens?

What rules must remain true?

What happens when it fails?

What happens if it happens twice?

What happens if the app is killed?

Where should this behavior live?
```

Then build.

QUICK START CHECKLIST

Use this whenever starting a new reference app.

```
PHASE 0
[ ] Choose reference app
[ ] Define scope
[ ] Define "done"

PHASE 1 — OBSERVE
[ ] Explore major screens
[ ] Explore major actions
[ ] Record loading states
[ ] Record empty states
[ ] Record errors
[ ] Investigate persistence
[ ] Investigate network behavior
[ ] Record unknowns

PHASE 2 — MAP
[ ] Identify capabilities
[ ] Identify relationships
[ ] Identify dependencies

PHASE 3 — JOURNEYS
[ ] Trace major user journeys
[ ] Identify data changes
[ ] Identify side effects

PHASE 4 — DOMAIN
[ ] Identify entities
[ ] Identify relationships
[ ] Identify ownership
[ ] Identify lifecycle

PHASE 5 — STATE
[ ] Identify states
[ ] Identify transitions
[ ] Identify invariants
[ ] Identify failure modes

PHASE 6 — DESIGN
[ ] Define boundaries
[ ] Define source of truth
[ ] Define data flow
[ ] Define persistence
[ ] Define synchronization
[ ] Define error handling
[ ] Define lifecycle behavior

PHASE 7 — FOUNDATION
```

```
[ ] Project structure
[ ] Networking
[ ] Persistence
[ ] DI
[ ] Authentication
[ ] Error handling
[ ] Logging
[ ] Testing infrastructure

PHASE 8 — FIRST SLICE
[ ] Pick important workflow
[ ] Build end-to-end
[ ] Test

PHASE 9 — VALIDATE
[ ] Review ownership
[ ] Review dependencies
[ ] Try adding a feature
[ ] Fix architectural problems

PHASE 10 — EXPAND
[ ] Build next vertical slice
[ ] Test
[ ] Review architecture
[ ] Repeat

PHASE 11 — HARDEN
[ ] Network failures
[ ] Lifecycle
[ ] Process death
[ ] Concurrency
[ ] Data failures
[ ] Authentication failures

PHASE 12 — FINISH
[ ] Performance
[ ] Security
[ ] Accessibility
[ ] Observability
[ ] Testing
[ ] Release
[ ] Maintainability review
```

ONE SENTENCE TO REMEMBER

> *Observe the behavior → understand the system → design ownership and boundaries → build one complete flow → validate → expand → harden.*